


UNIVERSITÀ DI PISA

# Generative and Deep Learning - Final Project

**Curriculum Artificial Intelligence**

**Giorgio Chelli**

**Track** 9**:** Implement two different generative models for graph data of your choice (e.g. autoregressive, diffusion-based, VAE, ...) and validate them on QM9 chemical graphs.

# 1 Introduction

## 1.1 Selected Models

Two generative models that implement the two opposing approaches of **one-shot generation** and **incremental generation** have been implemented, in order to understand the different implementation steps and the differences in the results.

**GraphVAE** : For the one-shot generation of molecules, a **GraphVAE** was implemented, trying to follow some of the suggestions in the paper [2].

**GRU** : For the incremental generation, on the other hand, an approach based on **SMILES fragmentation** has been defined to use an **RNN** (in this project, a **GRU**) to generate the molecules. For the SMILES fragmentation, first the fragmentation algorithm based on BRICS has been implemented, described in the paper [3], than SMILES Pair Encoding (SPE) [4], an adaptation of the Byte-Pair Encoding (BPE) algorithm [5] tailored for chemical sequences, has been adopted.

## 1.2 Dataset QM9

The QM9 dataset [1] is a dataset of molecules available in PyTorch's *torch.geometric* library. The dataset consists of $130,831$ molecules; each molecule has a maximum of $9$ heavy atoms (i.e. atoms other than hydrogen). There are five fundamental chemical elements: *Carbon* **(C)**, *Hydrogen* **(H)**, *Oxygen* **(O)**, *Nitrogen* **(N)** and *Fluorine* **(F)**. There are also four possible types of bonds: *Single, Double, Triple* or *Aromatic*.
The molecules are represented as graphs, where the nodes represent the atoms and the edges represent the bonds between them. Each molecule is also represented in SMILES format—that is, as a string—which is essential for natural language processing models applied to chemistry (such as fragment-based RNNs/GRUs or molecular Transformers).

## 1.3 RDKit

For the development of the project,the open-source **RDKit library** (*Open-Source Cheminformatics Software*), one of the leading tools for cheminformatics, molecular machine learning and chemical modelling, was used. RDKit allows you to manipulate, analyze and visualize chemical structures programmatically, and to carry out various operations on them.

# 2 GraphVAE

## 2.1 Pre-processing of data and dense representation of graphs

The implementation of the GraphVAE architecture required a pre-processing and standardization phase for the geometric data. As a one-shot generation model, GraphVAE requires tensors of a fixed, predefined size as input in order to correctly map the latent space.

As the QM9 dataset contains molecules with a variable number of heavy atoms (up to a maximum of 9), it was necessary to implement a zero-padding strategy to bring each structure back to a common dense representation:

- **Nodes representation**: A fixed-size vector of 9 elements, in which each cell contains the index associated with the type of atom. The positions corresponding to atoms not present in the molecule are set to $0$ (padding).
- **Bonds representation**: An adjacency matrix (9 x 9), a symmetric matrix in which the cell $(i, j)$ stores a value corresponding to the type of chemical bond between atom $i$ and atom $j$. Here too, the absence of bonds or interactions with padding nodes are encoded as zero.

To construct the dense graph representation for each molecule in QM9, node indices follow the native atom ordering assigned by RDKit during SMILES parsing. Since graphs are intrinsically invariant to node permutations, enforcing a rigid node ordering can introduce arbitrary biases during generation. To address this permutation ambiguity, during training we adopt a graph matching scheme based on the Hungarian algorithm [6], following the strategy in the GraphVAE paper [2].

## 2.2 Encoder structure

The implemented Encoder's architecture follow the structure of the encoder in the work [2], with some modern stabilization practices like LayerNorm instead of BatchNorm, Residual for gradient flow and Joint Pooling. It consists of three sequential levels of message passing, each followed by a normalization layer and the ReLU (Rectified Linear Unit) non-linear activation function.

At the convolution level, traditional **Graph Convolutional Networks (GCNs)** have not been used because they assign the same weight to every relationship. In the context of the QM9 dataset, this behaviour is biologically and chemically inefficient; we wish to weigh the impact of double, triple or aromatic bonds relative to single bonds.

For this reason, the architecture utilises layers of **Neural Network Convolution (NNConv**), that are one implementation of the **Edge-Conditioned Convolutions (ECC)** used in the original paper. Each NNConv module incorporates a linear neural network that takes the vector of bond features (edge attributes) as input and dynamically generates a customized weight matrix. In this way, the propagation of messages between atoms is explicitly conditioned by the type of chemical bond present.

To convert the feature matrices of the nodes into a single global molecular embedding vector, a strategy based on concatenating Global Mean Pooling and Global Max Pooling operations was implemented, while the original paper used only Mean Pooling. This approach proves to be optimal, as the two operators capture perfectly complementary topological information:

- **Global Mean Pooling:** This method allows the average global chemical composition to be calculated, but by its very nature tends to 'dilute' high-intensity local signals.
- **Global Max Pooling**: Extracts the maximum value for each feature across all nodes. In this way, it detects anomalies by isolating chemical peaks and salient functional groups that would otherwise be overlooked by average pooling.

## 2.3 Decoder structure

The decoder is responsible for generating the molecular graph structure from a compact latent vector $z$ (the one defined by the encoder during training, and a random value during generation). The process consists of two main stages: the characterization of the nodes (atoms) and the prediction of the bonds (edges).

The structure of the decoder is substantially different from the fully connected MLP to directly output flatten node and adjacency matrices, proposed in [2]. Such a dense architecture struggles to capture spatial and relational symmetries.

The latent vector $z$ is initially processed by a linear neural network (MLP) with normalization (LayerNorm) and activation (ReLU). The output is reshaped to produce an embedding matrix in which each of the 9 nodes is represented by a 64-dimensional vector. Next, a Multihead Attention module with a residual connection is applied, allowing the nodes to exchange information with one another, thereby capturing the global context of the molecule [8]. At the end of this stage, a linear layer predicts the type of atom associated with each node.

Edge features are computed by pairing node representations through concatenation and element-wise interaction, following the relational edge-prediction design introduced in MolGAN [7]. Once the nodes have been defined, the model must predict whether a link exists and, if so, its type for each pair $(i, j)$:

- **Pair construction:** A three-dimensional matrix is generated in which each cell $(i, j)$ contains the vectors of the source node and the destination node.
- **Feature fusion:** For each pair, the embeddings of node $i$ and node $j$ are concatenated with their element-wise product ($h_i \odot h_j$). This latter component acts as a non-linear operator that captures the chemical affinity and direct correlations between the two atoms.
- **Bond prediction:** The result is processed by an MLP, which returns the type of bond.

Finally, to obtain a symmetric matrix, one calculates the average of the predicted bond matrix and its transpose.

## 2.4 Loss

Standard cross-entropy, used in [2], suffers from extreme class imbalance due to graph padding and lacks any awareness of underlying chemical valency rules. In this work, similarly to MolGAN [7], which incorporates a chemical validity reward to bias the generation process toward plausible structures, our loss function enforces explicit valency constraints directly into the variational optimization objective. The loss function is defined as a linear combination of three components:

$$L_{total} = L_{nodes} + 2.0 \cdot L_{edges} + L_{valency} \tag{1}$$

- **Node loss**: The classification of the atoms associated with each node is optimized using the classic cross-entropy function.
- **Arcs loss**: The topology of the links is evaluated by calculating the cross-entropy for each pair of nodes $(i, j)$. Here, a weight matrix is applied to encouraging the model to connect the structure correctly.

- **Calculation of expected valence**: This component introduces regularization based on real chemical constraints. By multiplying the bond probabilities (obtained via Softmax) by the respective chemical bond orders (None = $0$; Single = $1$; Double = $2$; Triple = $3$; Aromatic = $1.5$), the model estimates the predicted valence for each atom. The predicted valence is compared with the maximum theoretical valence permitted for that specific target atom (e.g. Carbon = $4$, Nitrogen = $3$, Oxygen = $2$, Fluorine = $1$, Padding = $0$). Using the ReLU function, the model applies a penalty in case of **over-valence**; If the predicted valence exceeds the maximum permitted value, the error is heavily penalized to avoid chemically impossible atoms.

## 2.5 Graph Alignment via Hungarian Algorithm

To calculate the reconstruction loss in graph generative models, one must account for graph isomorphism and permutation invariance. Following the GraphVAE framework [2], we align the predicted node and edge topologies with the target ground-truth graphs using the Hungarian Algorithm.

The optimal permutation matrix $P^*$ for a graph with $K$ potential nodes is computed by minimizing a bi-criterion assignment cost tensor $C \in \mathbb{R}^{K \times K}$:

$$C_{i,j} = \alpha \cdot C_{i,j}^{\text{nodes}} + \beta \cdot C_{i,j}^{\text{edges}} \tag{2}$$

where $C_{i,j}^{\text{nodes}}$ measures the cross-entropy loss between the predicted features of $i$-th node and $j$-th ground-truth node:

$$C_{i,j}^{\text{nodes}} = -\log P(\hat{X}_i = X_j^{\text{target}}) \tag{3}$$

Similarly, $C_{i,j}^{\text{edges}}$ computes the structural discrepancy across candidate edge couplings with neighboring nodes.

**Implementation and Training Stability ($\beta = 0$):** While the general formulation incorporates both node and edge terms, setting $\beta > 0$ introduces instability because edge logits initially contain unstructured noise, including $C^{\text{edges}}$ causes label flipping—forcing the matching algorithm to misalign atomic identities.

To enforce stable convergence and preserve atomic semantics, we adopt $\beta = 0.0$ during training. Once $P^*$ is derived via `scipy.optimize.linear_sum_assignment`, the predicted node and edge logit tensors are permuted accordingly prior to backpropagation:

$$\hat{X}_{\text{aligned}} = P^* \hat{X}, \quad \hat{A}_{\text{aligned}} = P^* \hat{A} (P^*)^T \tag{4}$$

## 2.6 Training

The GraphVAE was trained using the following settings:

- **Latent Vector Size** = $64$, represents the dimensionality of the continuous vector space into which the encoder compresses the molecular topology. A dimension of $64$ was chosen as an optimal compromise: too low values would limit the model's ability to encode complex graphs (causing underfitting), while too high values would reduce the regularization effect of the variational autoencoder, facilitating overfitting.

- **Embedding Vector Size** = $256$: This refers to the dimensionality of the hidden linear layers used in the encoder and decoder architecture. This parameter governs the computational and expressive capacity of the internal feed-forward neural networks, allowing the extraction of complex nonlinear features from the latent vector.
- **Maximum KL divergence weight** = $0.1$: The $\beta$ parameter controls the balance within the VAE loss function. A maximum value limited to $0.1$ (by adopting a *KL-annealing* protocol during the first epochs) was introduced to avoid the *posterior collapse* phenomenon.
- **Size of internal nodes** = $64$: The size of the internal embedding vector associated with each atom within the decoder, used by the Multi-head Attention module to calculate mutual interactions and refine bond prediction.
- **Heads** = $4$: Allows the network to simultaneously project the node embedding into four different subspaces. This way, during the decoding phase, each atom can "pay attention" and calculate bonding relationships with other atoms according to different and parallel chemical-geometric criteria.

The training was executed for up to $180$ epochs on the QM9 dataset, which was split into a training set (80%) and a validation set (20%), with the batch size set to 32.

The Adam algorithm was used for optimization, with an initial learning rate of $5 \times 10^{-4}$. The learning rate is halved (ReduceLROnPlateau) if the validation loss does not improve for 5 consecutive epochs. Finally, to prevent overfitting, an early stopping strategy with a patience of 10 epochs has been implemented, saving the optimal weights of the encoder and decoder as soon as the minimum validation loss is recorded.

Furthermore, as the KL-Vanishing problem arose during the initial training stages, a Beta-Annealing technique was employed, in which the coefficient $\beta$ associated with the Kullback-Leibler Divergence (KLD) is increased linearly from 0 to 0.1 over the first 20 epochs. This allows the model to focus initially on the quality of the molecular reconstruction and, only subsequently, on the regularization of the latent space.

### 2.6.1 Training results

The learning curve of the GraphVAE model (Figure 1) shows an optimal convergence profile. In the early epochs, a sharp drop in both training and validation loss indicates that the model rapidly learns the core distribution of atomic types and basic adjacency features of the QM9 dataset. Subsequently, the trajectory stabilizes into a steady trend, reflecting the gradual refinement of complex edge topologies and valency constraints within the latent space. The stable gap between the two curves and the absence of overfitting demonstrate GraphVAE's solid generalization capabilities.

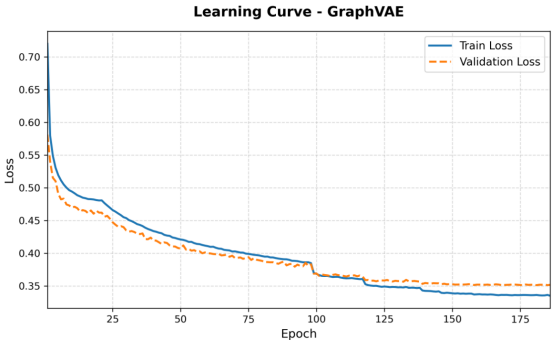



Figure 1: GraphVAE Learning Curve

## 2.7 Generation

New molecular structures are generated by sampling directly from the latent space and converting the resulting graphs into SMILES strings using the RDKit library. $n_samples$ latent vectors are generated, drawn at random from a standard Gaussian distribution. The decoder processes these vectors, returning the logits associated with nodes and edges. The logits are converted into probabilities using a Softmax function and sampled from a multinomial distribution.

To limit the tendency of GraphVAE to create molecules as multiple disconnected fragments, we adopted the strategy used in the original GraphVAE ([2]) paper, in which connectivity is enforced during testing by building a **maximum spanning tree (MST)** over the set of probable nodes, using the edge probabilities as similarity weights.

A valence-sensitive spanning tree was implemented, in which each forced edge is added only if it does not exceed the residual valence capacity of both terminal atoms; if no chemically compatible bond type is available, the edge is left unconnected rather than forced, and the molecule is then discarded.

The resulting real atoms and bonds are assembled into an editable molecular object. As the generation is probabilistic, the raw structures undergo a process of post-processing and chemical sanitisation.

# 3 GRU

For the implementation of the incremental generative model, a **Gated Recurrent Unit (GRU)** network was chosen. This choice was motivated by the small size of the molecules under investigation (maximum 9 heavy atoms): in cases involving short sequences, such as this one, sequential recurrent models do not suffer significantly from the problem of information loss or gradient degradation along the sequence and allow for faster convergence.

To process the molecules using the GRU, the structures were initially converted into SMILES strings and subsequently processed. Two different and contrasting approaches were used for fragmentation:

- **Fragmentation Algorithm based on BRICS** [3] [9]
- **SMILES Pair Encoding (SPE)** [4]

## 3.1 Fragmentation Algorithm based on BRICS

Directly applying BRICS fragmentation [9] to the QM9 dataset revealed significant structural limitations. BRICS relies on a predefined set of cleavage rules. QM9 consists exclusively of small, highly constrained molecules; most structures lack the target functional groups required by BRICS rules.

As a result, the algorithm failed to decompose a significant portion of the QM9 dataset into smaller units, returning intact molecules as single, large tokens. This generated an excessively sparse fragment vocabulary with a severe class imbalance, severely hampering the learning ability of the autoregressive GRU algorithm.

## 3.2 Smiles Pair Encoding (SPE)

To overcome the strict rule-based limitations imposed by BRICS on small molecule datasets, Smiles Pair Encoding (SPE) [4] was implemented. SPE adapts the Byte Pair Encoding (BPE) algorithm specifically for SMILES representations, iteratively merging the most frequent pairs of adjacent characters or subsequences into unified tokens.

SPE is entirely data-driven, this offers three crucial advantages:

1. **Guaranteed coverage:** Unlike BRICS, which leaves non-cleavable QM9 molecules intact, SPE handles any arbitrary structure seamlessly, falling back on single-character tokens if an unseen pattern is encountered.

2. **Optimal sequence length:** By compressing atomic groups that frequently occur together (e.g., ring structures or common functional groups) into single vocabulary tokens, SPE reduces the effective length of the sequence processed by the GRU decoder.

3. **Controlled vocabulary size:** SPE allows explicit regulation of vocabulary size. The vocabulary size selected for QM9 was empirically selected as $100$ tokens.

## 3.3 GRU implementation

The GRU is implemented in three main stages:

- **Embedding Layer**: It takes the vectors of numerical tokens as input and maps each fragment index to a continuous, dense vector space of fixed dimension.

- **GRU**: The GRU network processes the embedding sequence step by step while maintaining an internal hidden state. The use of the GRU eliminates the need for positional encoding modules, as the sequential nature of the recurrent network natively encodes the temporal order and position of the fragments within the sequence.

- **Linear Classification**: The output produced by the GRU for each time step is projected by a final linear layer into the vocabulary-dimensional space. This layer produces the logits required to calculate the probability of generating the next token using a standard cross-entropy loss function.

## 3.4 Training

For training GraphGRU, the key hyperparameters of the network and the optimization process are:

- **Token Embedding Dimension** = $256$: Represents the dimensionality of the continuous space into which each character of the vocabulary is projected before being processed by the recurrent layers.

- **Hidden State Dimension** = $512$: Represents the dimensionality of the internal state vector (memory) of the GRU units. A value of $512$ provides the network with the expressive capacity needed to memorize complex long-range dependencies.

- **Number of Recurrent Layers** = 3: Specifies the depth of the GRU architecture. The first layers capture the local short-range syntax, while the upper layers synthesize the global topological structure of the molecule.
- **Dropout Rate** = 0.2: Prevents overfitting on specific SMILES strings in the training dataset.

The dataset was split into a training set (80%) and a validation set (20%), with the batch size set to 64. Cross-entropy loss was used to guide the autoregressive learning process; this explicitly excludes padding tokens from the gradient calculation.

The optimization process utilizes the Adam algorithm with an initial learning rate of $10^{-3}$.

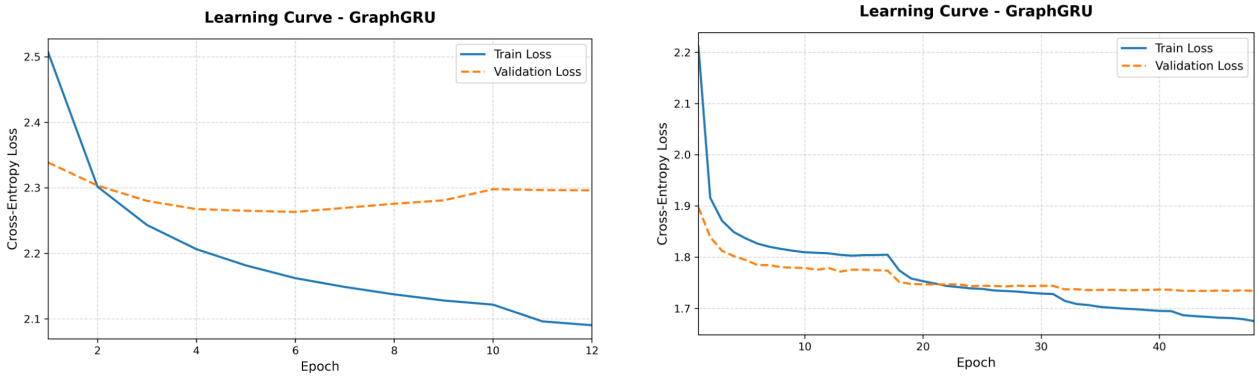
The training cycle, run for a maximum of 50 epochs, constantly monitors the loss on the validation set. The ReduceLROnPlateau scheduler dynamically halves the learning rate after 5 consecutive epochs of stagnation. Finally, an early stopping mechanism terminates training if no improvements in the validation loss are observed for 10 consecutive epochs.

### 3.4.1 Training results

Figure 2 illustrates the critical impact of the chosen tokenization scheme on the generalization of the model.

When trained using BRICS fragmentation (Figure 2a), the model shows rapid overfitting: while the training loss steadily decreases, the validation loss reaches its minimum around Epoch 6 ($\sim 2.26$) and subsequently diverges upwards. This behavior directly reflects the vocabulary shortage caused by the BRICS on QM9; the autoregressive decoder stores entire undecomposed molecular structures rather than learning generalized substructural assembly rules.

In contrast, the SMILES Pair Encoding (SPE) representation (Figure 2b) demonstrates superior training stability and generalization capabilities. The validation loss strictly follows the training trajectory, reaching a lower baseline and maintaining a stable plateau. This empirical evidence confirms that data-driven subsequence tokenization via SPE effectively addresses dataset-specific fragmentation bottlenecks in recurrent generative architectures.




(a) GraphGRU with BRICS Tokenization

(b) GraphGRU with SPE Tokenization

Figure 2: Comparison of training and validation Cross-Entropy Loss trajectories between BRICS and SPE tokenization strategies

### 3.5 Generation

The autoregressive generation pipeline evaluates candidate models by sampling token sequences from the learned conditional probability distribution using temperature-scaled multinomial sampling. However, reconstructing a valid molecular graph from the generated token sequence reveals key operational differences between the BRICS and SPE tokenization schemes.

**Bottleneck in BRICS reconstruction** As implemented in the BRICS framework, tokens represent discrete molecular fragments with wildcards (`*`) representing open attachment points. Consequently, the generation cycle requires continuous monitoring of accumulated heavy atom counts to prevent sequence overflow.

More critically, graph reconstruction (`reconstruct_smiles`) requires explicit molecular editing operations via RDKit:

1. Pairwise identification and alignment of dummy atoms between consecutive fragments.
2. Check bond type compatibility across fragment boundaries.

**Reconstruction via SPE** In contrast, SMILES Pair Encoding (SPE) delegates structural coherence directly to the autoregressive model. Since SPE subwords represent contiguous SMILES substrings, the reconstruction reduces to a deterministic concatenation of strings. The resulting SMILES string is passed directly to RDKit's `SanitizeMol` parser to check for syntactic validity.

## 4 GraphVAE vs GRU Generation results

| Model | Temp. ($T$) | Validity (%) | Uniqueness (%) | Novelty (%) |
|---|---|---|---|---|
| **GraphVAE** | 0.5 | $51.14 \pm 1.99$ | $99.36 \pm 0.44$ | $93.78 \pm 0.18$ |
| | 1.0 | $51.52 \pm 1.63$ | $99.65 \pm 0.23$ | $94.83 \pm 0.99$ |
| | 1.5 | $50.72 \pm 1.82$ | $99.73 \pm 0.19$ | $94.46 \pm 0.26$ |
| **GraphGRU (BRICS)** | 0.5 | $60.52 \pm 0.80$ | $98.71 \pm 0.39$ | $1.28 \pm 0.42$ |
| | 1.0 | $52.48 \pm 0.35$ | $88.07 \pm 0.74$ | $10.60 \pm 2.09$ |
| | 1.5 | $40.52 \pm 0.82$ | $85.49 \pm 0.95$ | $33.54 \pm 2.29$ |
| **GraphGRU (SPE)** | 0.5 | $\mathbf{83.92 \pm 0.64}$ | $95.61 \pm 0.41$ | $3.69 \pm 0.35$ |
| | 1.0 | $\mathbf{75.62 \pm 1.27}$ | $\mathbf{99.63 \pm 0.15}$ | $17.09 \pm 0.80$ |
| | 1.5 | $\mathbf{61.04 \pm 1.85}$ | $\mathbf{99.90 \pm 0.13}$ | $\mathbf{33.05 \pm 1.36}$ |

Table 1: Generative performance comparison across all evaluated models.

**Validity** GraphVAE achieves a validity in line with the original paper ([2]), stable across different temperatures (≈51%). GraphGRU with SPE achieves substantially higher validity at low temperature (83.92%), demonstrating the advantage of autoregressive and string-level consistent construction over one-shot decoding of dense graphs: because SPE subwords directly concatenate into syntactically valid SMILES fragments, the graph assembly errors that limit GraphVAE

do not occur. BRICS lags behind the other two configurations, consistent with its coarser fragment vocabulary.

**Uniqueness** All configurations except BRICS at $T \geq 1.0$ maintain uniqueness $\approx 95\%$, indicating that the latent space/sequence space coverage is extensive and none of the models suffer from severe modal collapse issues. BRICS's degradation in uniqueness at higher temperatures (from 88.07% to 85.49%) reflects its sparse and unbalanced fragment vocabulary, which biases sampling toward a limited set of frequently occurring tokens.

**Novelty** The three models diverge markedly on novelty. GraphGRU's novelty follows the expected temperature scaling behavior: low temperatures focus sampling on high-probability token transitions, typical of training, while higher temperatures prioritize exploration at the expense of validity. GraphVAE's novelty, in contrast, remains uniformly high ($\approx 94 - 95\%$) across all temperatures and substantially exceeds the range reported for the original architecture. The reason for this result can be found in the analysis of the Wasserstein distance (Sect. 4) and represents an intrinsic limitation of one-shot decoding of dense graphs.

**Wasserstein distance analysis** Wasserstein distances (Table 2) allow us to explain the novelty results obtained between the two different models:

- **GraphGRU (SPE)** achieves the lowest Wasserstein distances for almost all descriptors, the closest alignment with QM9 observed in this study. This result is achieved despite a non-negligible novelty rate ($\approx 33\%$ at T = 1.5), indicating that the novelty of SPE reflects genuine exploration within a chemically realistic distribution. The model generates previously unseen molecules that respect the structural statistics that characterize QM9.
- **GraphGRU (BRICS)** exhibits the opposite pattern: despite very low novelty at low temperatures, its Wasserstein distances remain relatively sparse. The low novelty in this case is not an indicator of distributional fidelity, but rather of memorization: the model overwhelmingly reproduces fragments nearly identical to the BRICS vocabulary, while the fragments it recombines novelly tend to distort the overall molecular weight and polarity profile.
- **GraphVAE** occupies a third distinct regime: high novelty occurs in conjunction with the largest Wasserstein distance on the ring count and a substantially higher TPSA distance compared to GraphGRU-SPE. This distributionally confirms that GraphVAE's novelty is partly driven by structurally atypical generation rather than by the exploration of regions of molecular space chemically consistent with QM9.

Overall, these results indicate that novelty and uniqueness alone are not sufficient to assess generative quality: a high rate of novelty is significant only when combined with low distributional divergence from the reference dataset. By this joint criterion, GraphGRU with SPE tokenization is the only model in this study whose novelty can be interpreted as a genuine chemical generalization rather than as a symptom of the corresponding failure mode observed in each generation approach.

| Model | MW | LogP | TPSA | QED | NumRings |
|---|---|---|---|---|---|
| GraphVAE | 2.3427 | **0.1670** | 3.2496 | **0.0076** | 0.3226 |
| GraphGRU (BRICS) | 11.7337 | 0.3966 | 4.7542 | 0.0140 | 0.2351 |
| GraphGRU (SPE) | **0.4056** | 0.2231 | **1.3196** | 0.0117 | **0.0590** |

Table 2: Wasserstein distance between generated and QM9 training distributions across five physicochemical descriptors (lower is better). Best result per descriptor in bold.

### 4.1 Aromatic Bond Collapse

Examining the high novelty of GraphVAE, we found that the model systematically underproduces aromatic bonds on heteroatoms. This shortcoming stems from the decoder's independent predictions for each individual edge: aromatic rings require simultaneous coherence across 5-6 edges to pass RDKit's kekulization check, which independent sampling for each edge rarely achieves, even when individual bond probabilities are well-calibrated. This limitation is not specific to our implementation: the original formulation of GraphVAE explicitly assumes independence among all predicted variables [2]. This same mechanism also explains the anomalous ring count distribution reported in Section 4, since ring count directly depends on multi-edge coherence.

**Computational efficiency and convergence dynamics** GraphVAE requires full execution over 180 epochs. Although training is more computationally demanding, it produces a continuous, uniform representation capable of rapid parallel sampling during inference. Both GraphGRU variants incorporated early stopping based on validation loss, producing distinctly different convergence profiles. The BRICS model ended prematurely after just 12 epochs. This rapid stop reinforces the findings on overfitting and low novelty; the rigid, high-granularity macros allowed the network to quickly memorize token transitions rather than learning a generalized distribution. SPE converges naturally after 50 epochs. The subword vocabulary is long enough to prevent vocabulary memorization, but significantly faster and lighter to train than matrix-based graphical decoders.

## 5 Conclusions

In summary, GraphGRU with SPE emerges as the most balanced and effective framework for valid and constrained molecular design, while GraphVAE serves as a highly capable exploratory baseline for continuous latent space optimization. GraphVAE structurally suffers from the lack of explicit connectivity constraints in the decoder, a limitation widely documented in the literature for models operating on dense adjacency matrices.

More broadly, this work suggests that novelty and uniqueness alone are insufficient to assess generative quality: a high novelty rate is meaningful only when paired with low distributional divergence from the reference dataset (Section 4). By this joint criterion, only GraphGRU-SPE's novelty reflects genuine chemical generalization, while GraphVAE's stems in part from a structural bias – the aromatic bond collapse discussed above – rather than superior exploration of chemically valid molecular space.

# References


[1] Raghunathan Ramakrishnan, Pavlo O. Dral, Matthias Rupp, and O. Anatole von Lilienfeld. Quantum chemistry structures and properties of 134 kilo molecules. *Scientific Data*, 1:140022, 2014.

[2] M. Simonovsky and N. Komodakis, "GraphVAE: Towards Generation of Small Graphs Using Variational Autoencoders," 2018. [Online]. Available: `https://arxiv.org/pdf/1802.03480`.

[3] M. Podda, D. Bacciu, and A. Micheli, "A Deep Generative Model for Fragment-Based Molecule Generation" 2020. [Online]. Available: `https://arxiv.org/pdf/2002.12826`.

[4] Xinhai Li and De-en Jiang. SMILES Pair Encoding: A natural language processing-based tokenization approach for chemical sequences. *ChemRxiv*, 2021.

[5] Rico Sennrich, Alexandra Birch, and Barry Haddow. Neural Machine Translation of Rare Words with Subword Units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics*, pages 1715–1725, 2016.

[6] Harold W. Kuhn. The Hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2(1-2):83–97, 1955.

[7] Nicola De Cao and Thomas Kipf. MolGAN: An implicit generative model for small molecular graphs. *arXiv preprint arXiv:1805.11973*, 2018.

[8] Vijay Prakash Dwivedi and Xavier Bresson. A generalization of transformer networks to graphs. *arXiv preprint arXiv:2012.09699*, 2020.

[9] Jörg Degen, Jörg Wegner, Lothar Weisel, Andreas Christ, and Matthias Rarey. On the art of compiling and using conceptual building blocks for medicinal chemistry design. *ChemMedChem*, 3(10):1503–1507, 2008.